

UFUG 2602 数据结构与算法设计

曹北彦、常腾飞

智能工程学院与物联网学域
香港科技大学（广州）

回顾：

- 数据结构（表示/组织） + 操作 = 数据类型
- 数据类型
 - 基本数据类型：由编程语言提供
 - 用户自定义数据类型：程序员按需定义
- 用户自定义数据类型
 - 可以建立在其他类型之上（基本类型或用户自定义类型）。
- 抽象数据类型
 - 两处分离：
 - 数据值的定义与其表示相分离。
 - 操作的定义与其实现相分离。
 - 封装：隐藏实现的内部细节。

```

1 # Tic-Tac-Toe Game
2 class TicTacToe:
3     '''
4     Management of a Tic-Tac-Toe Game
5     No strategy
6     '''
7     def __init__(self):
8         ''' start a new game'''
9         self._board = [ [' ' ] * 3 for j in range(3) ]
10        self._player = 'X'
11
12    def mark(self, i, j):
13        ''' Put X or O mark at position (i,j) for next player's turn '''
14
15        if not(0 <= i <= 2 and 0 <= j <= 2):
16            raise ValueError('Invalid board position')
17        if self._board[i][j] != ' ':
18            raise ValueError('Board position occupied')
19        if self.winner() is not None:
20            raise ValueError('Game is already complete')
21        self._board[i][j] = self._player
22
23        if self._player == 'X':
24            self._player = 'O'
25        else:
26            self._player = 'X'
27
28    def _is_win(self, mark):
29        ''' Check whether the board configuration is a win for a given player'''
30        board = self._board
31
32        return (mark == board[0][0] == board[0][1] == board[0][2] or # row 0
33                mark == board[1][0] == board[1][1] == board[1][2] or # row 1
34                mark == board[2][0] == board[2][1] == board[2][2] or # row 2
35                mark == board[0][0] == board[1][0] == board[2][0] or # column 0
36                mark == board[0][1] == board[1][1] == board[2][1] or # column 1
37                mark == board[0][2] == board[1][2] == board[2][2] or # column 2
38                mark == board[0][0] == board[1][1] == board[2][2] or # diagonal
39                mark == board[0][2] == board[1][1] == board[2][0]) # rev diag
40
41    def winner(self):
42        ''' return mark of the winning player, or None to indicate a tie'''
43        for mark in 'XO':
44            if self._is_win(mark):
45                return mark
46        return None
47
48    def __str__(self):
49        ''' return th estring representation of the current game board'''
50        rows = ['|'.join(self._board[r]) for r in range(3)]
51        return '\n-----\n'.join(rows)
52

```

```

53
54 game = TicTacToe()
55
56 # X moves          # O moves
57 game.mark(1,1);    game.mark(0,2)
58 game.mark(2,2);    game.mark(0,0)
59 game.mark(0,1);    game.mark(2,1)
60 game.mark(1,2);    game.mark(1,0)
61 game.mark(2,0);
62
63 print(game)
64 winner = game.winner()
65 if winner is None:
66     print('Tie')
67 else:
68     print(winner, 'wins')
69

```

```

O|X|O
-----
O|X|X
-----
X|O|X
Tie

```

随堂练习：通用的获胜判定

- 改写 `_is_win(mark)`，不要写死下标，改为对二维数组循环。
- 解法应当：
 - 检查所有行
 - 检查所有列
 - 检查两条对角线
- 扩展（选做）
 - 改成对任意棋盘大小 N 都适用

提示

- 「是否存在某一行，所有列都是同一个标记？」

```
for row in self._board:
    if all(cell == mark for cell in row):
        return True
```

- 固定列下标、变化行下标，就得到一列。

```
for col in range(3):
    if all(self._board[row][col] == mark for row in range(3)):
        return True
```

- 主对角线、副对角线，利用下标关系：

利用下标关系：

主对角线 $\rightarrow i == j$

副对角线 $\rightarrow i + j == N - 1$

示例实现

```
def _is_win(self, mark):
    """检查当前棋盘是否为某玩家获胜。"""
    board = self._board
    N = len(board)

    # 检查行
    for r in range(N):
        if all(board[r][c] == mark for c in range(N)):
            return True

    # 检查列
    for c in range(N):
        if all(board[r][c] == mark for r in range(N)):
            return True

    # 检查主对角线
    if all(board[i][i] == mark for i in range(N)):
        return True

    # 检查副对角线
    if all(board[i][N - 1 - i] == mark for i in range(N)):
        return True

    return False
```

提纲

- 用查找来举例
- 算法分析
 - 什么是好算法?
 - 伪代码
 - 最好、最坏与平均情况分析
 - 插入排序
 - 二分查找
 - 渐近分析

找出糖水

🤔有 100 杯水，其中只有一杯是糖水。怎样尽快找到它？

➤可能的做法：朴素方法（暴力）
😞一杯一杯尝 → 最多需要 100 次

➤能否做得更好？
➤有没有更聪明、更快的办法？



示例

- 假设我们用整数列表作为数据结构，来存储集合 S （含 n 个整数）。
- 考虑如何支持查找操作：
 - 函数 `search(S, search_num)` 返回 `search_num` 在列表中的下标 i ；若未找到则返回 -1。
- Example:
 - $S = \{3, 7, 9, 12, 13, 18, 20, 23, 27\}$
 - Is 20 in S ?
- 我们给出两种实现算法。

示例：线性查找

- 直接读取下标为 i 的整数，对每个 $i \in [0, n - 1]$ 。
- 若存在 i 使得 $S[i] = \text{search_num}$ ，则返回 i ；
- 否则，返回 -1。

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

示例：二分查找

- 假设整数集合已排序。
- 将 `search_num` 与位于中间的元素（第 $(n/2)$ 个，来自 S ）进行比较）

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

查找值 = 18 规模 = 9

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

左=0

右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=4

左=5

右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=6

左=5 右=5

右=5

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=5

查找值 = 28 规模 = 9

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

左=0

右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=4

左=5

右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=6

左=7 右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=7

左=8 右=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

中=8

线性查找 vs 二分查找：哪个更好？

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

比较之前：先做个预测

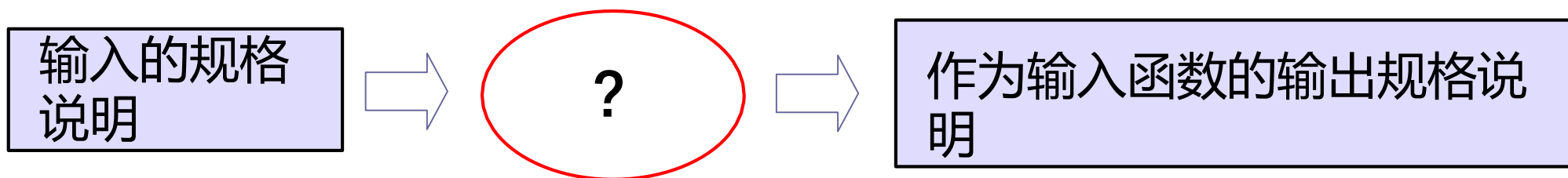
- 假设：
 - 数组大小 = 1,000
 - 目标不在数组中
- **问题：**
 - 线性查找要比较多少次？
 - 二分查找要比较多少次？

提纲

- 用查找来举例
- **算法分析**
 - 什么是好算法?
 - 伪代码
 - 最好、最坏与平均情况分析
 - 渐近分析

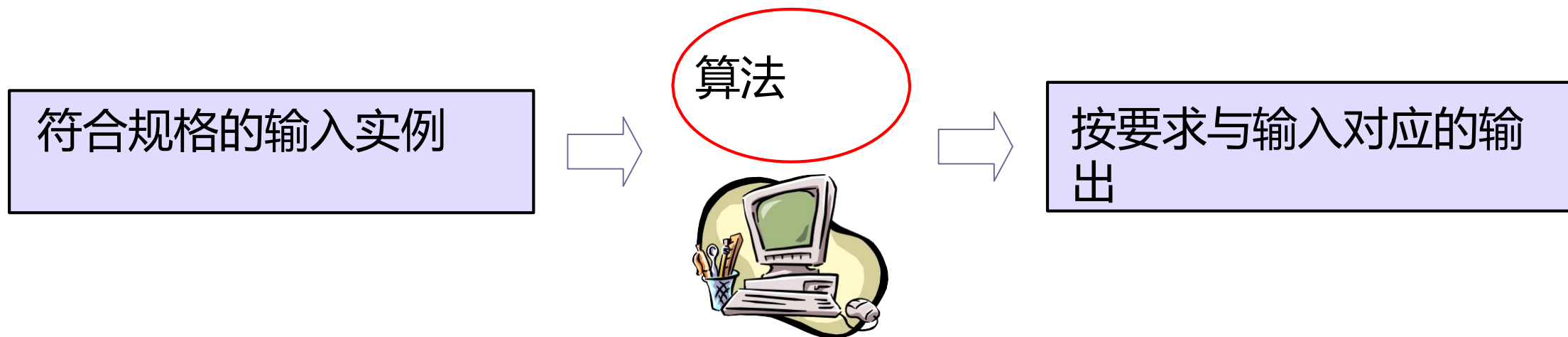
算法问题

- 满足规格说明的输入实例有无穷多个。
 - 例如：非零有限长度、非递减的自然数有序序列：
 - 1, 20, 908, 909, 100000, 1000000000.
 - 3.



算法解

- 算法描述对输入实例采取的动作
- 同一算法问题可以有許多正确算法



什么是好算法？

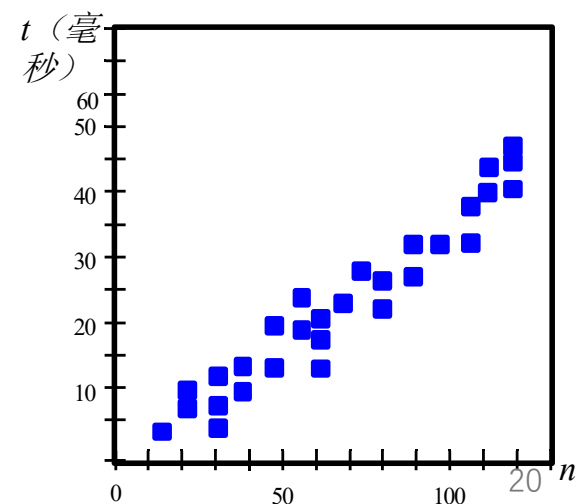
- 高效：
 - 运行时间
 - 占用空间
- 效率作为输入规模的函数：
 - 输入数字的比特数
 - 数据元素个数（数字、点等）

度量运行时间

应该如何度量算法的运行时间？

实验研究

- 写出实现该算法的程序
- 用不同规模、不同组成的数据集运行程序。
- 用系统调用精确测量实际运行时间。



实验研究的局限

- 必须实现并测试算法，才能知道运行时间。
- 实验只能覆盖有限的输入集合，
 - 未必能代表实验未包含的那些输入上的运行时间。
- 比较两个算法时，需要相同的软硬件环境

超越实验研究

我们将建立一套分析算法运行时间的一般方法。该方法

- 使用算法的高层描述，而不是去测试某一种实现。
- 考虑所有可能的输入。
- 能以独立于软硬件环境的方式评价任意算法的效率。

例子

- **算法** `arrayMax(A, n)`:
 - 输入：存放 n 个整数的数组 A 。输出： A 中的最大元素。

伪代码（命令式）

- 自然语言与高层编程概念的混合，用来描述数据结构或算法的一般实现思路。

- 例如 算法 `arrayMax(A, n)`:

输入：存放 n 个整数的数组 A 。

输出： A 中的最大元素。

`currentMax  $\leftarrow$  A[0]`

`for  $i \leftarrow 1$  to  $n-1$  do`

`if  $\text{currentMax} < A[i]$  then  $\text{currentMax} \leftarrow A[i]$`

`return currentMax`

伪代码

比普通散文更有结构，又不如编程语言那么形式化

- 表达式：
 - 用标准数学符号描述数值和布尔表达式
 - 用 $\leftarrow$ 表示赋值 (Java 中的 ' $=$ ')
 - 用 $=$ 表示相等关系 (Java 中的 ' $==$ ')
- 方法声明：
 - **算法** name(param1, param2)

伪代码

常见有若干种风格。

- 程序构造：
 - 分支结构：if ... then ... [else ...]
 - while 循环：while ... do
 - repeat 循环：repeat ... until ...
 - for 循环：for ... do
 - 数组下标：A[i]、A[i,j]
- 方法：
 - 调用：object.method(args)
 - 返回：return value

算法问题：找第二大元素

- **输入：**含 n 个整数的数组 A ，其中 $n \geq 2$
- **输出：**数组中的第二大元素
- **约束：**
 - 不允许对数组排序
 - 算法应对任意合法输入都正确
- **例子：**

输入：[3, 10, 5, 8, 2]

输出：8

算法分析

- **基本操作：与编程语言无关的底层操作。**
- 可以在伪代码中识别。例如：
 - 数据移动（赋值）
 - 控制（分支、子程序调用、返回）
 - 算术与逻辑运算（如加法、比较）
- 通过审阅伪代码，可以统计算法执行的基本操作次数。

整理手中的牌

🤔你手里有一副顺序打乱的扑克牌。

- 问1：怎样高效地把它们排成升序？
- 问2：新牌一张张发来时，应如何插入到正确位置？



另一例子：排序

输入

数字序列

$a_1, a_2, a_3, \dots, a_n$

2 5 4 10 7



输出

该序列的一个
数字序列

$b_1, b_2, b_3, \dots, b_n$

2 4 5 7 10

正确性（对输出的要求）

对任意给定输入，算法停机并给出输出：

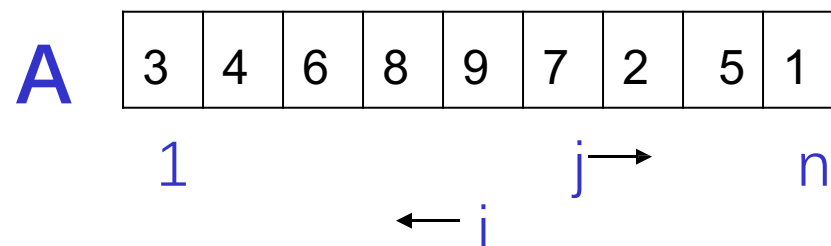
- $b_1 < b_2 < b_3 < \dots < b_n$
- $b_1, b_2, b_3, \dots, b_n$ 是 $a_1, a_2, a_3, \dots, a_n$ 的一个排列

运行时间

取决于

- 元素个数 (n)
- 它们已经（部分）有序的程度
- 算法

插入排序



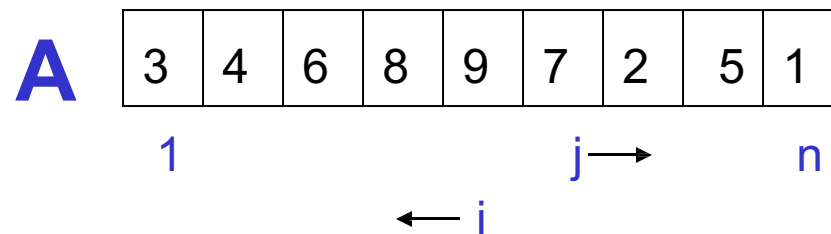
策略

- 从「空手」开始
- 把一张牌插入已经排好序的手牌的正确位置
- 直到所有牌都插入/排好

输入: $A[0..n-1]$ — 整数数组

输出: A 的一个排列, 使 $A[0] \leq A[1] \leq \dots \leq A[n-1]$

插入排序



策略

- 从「空手」开始
- 把一张牌插入已经排好序的手牌的正确位置
- 直到所有牌都插入/排好

输入：A[0..n-1] — 整数数组；输出：A 的排列，使 $A[0] \leq A[1] \leq \dots \leq A[n-1]$

```
for j ← 1 to n-1 do
    key ← A[j]
    // 把 A[j] 插入已排序序列
    A[0..j-1]
    i ← j-1
    while i ≥ 0 and A[i] > key
        do A[i+1] ← A[i] i--
    A[i+1] ← key
```

插入排序的分析

```
for j ← 1 to n-1 do
  key ← A[j]
  // 把 A[j] 插入已排序的
  // 序列 A[0..j-1]
  i ← j - 1
  while i ≥ 0 and A[i] > key
    do A[i+1] ← A[i]
       i--
  A[i+1] ← key
```

代价	次数
----	----

c_1	n
-------	-----

c_2	$n-1$
-------	-------

0	$n-1$
---	-------

c_3	$n-1$
-------	-------

c_4	$\sum_{j=1}^{n-1} t_j$
-------	------------------------

c_5	$\sum_{j=1}^{n-1} (t_j - 1)$
-------	------------------------------

c_6	$\sum_{j=1}^{n-1} (t_j - 1)$
-------	------------------------------

c_7	$n-1$
-------	-------

t_j : 插入 $A[j]$ 时 while
循环执行的次数

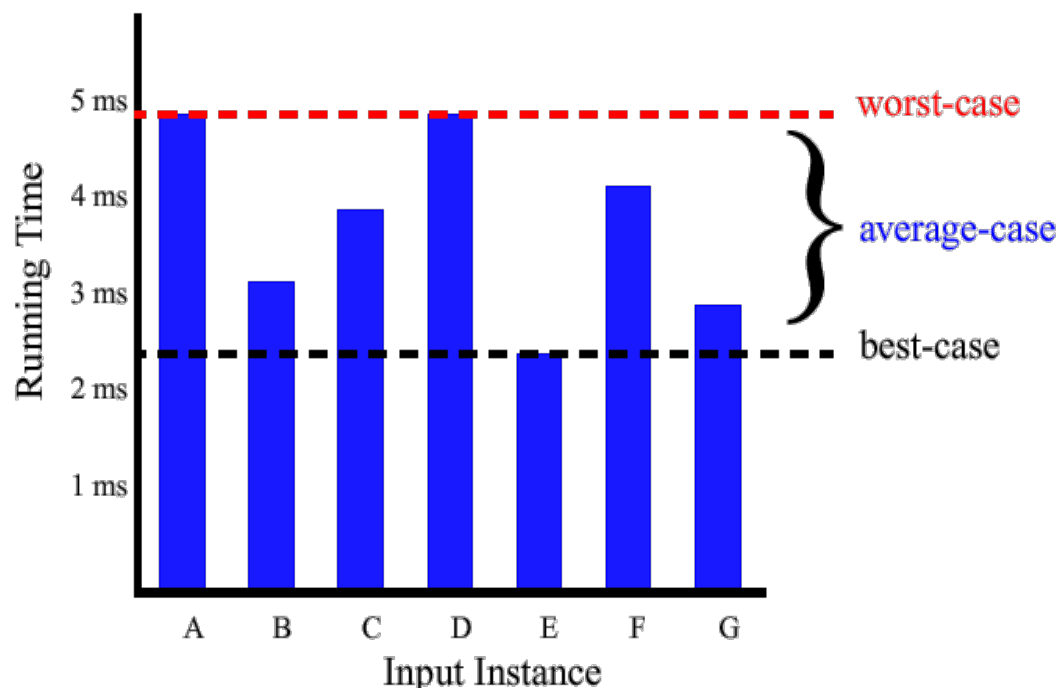
$$\begin{aligned} \text{Total time} = & n \cdot (c_1 + c_2 + c_3 + c_7) + \sum_{j=1}^{n-1} t_j \cdot (c_4 + c_5 + c_6) \\ & - (c_2 + c_3 + c_7) - (c_5 + c_6) \cdot (n - 1) \end{aligned}$$

最好/最坏/平均情况 (1)

- Total time = $n \cdot (c_1 + c_2 + c_3 + c_7) + \sum_{j=1}^{n-1} t_j \cdot (c_4 + c_5 + c_6) - (c_2 + c_3 + c_7) - (c_5 + c_6) \cdot (n - 1)$
- **Best case:**
 - 元素已经有序; $t_j = 1$; 运行时间的量级为 n
- **Worst case:**
 - 元素按逆序排列; $t_j = j + 1$; 运行时间的量级为 n^2
- **Average case:**
 - $t_j = (j + 1)/2$; 运行时间的量级为 n^2

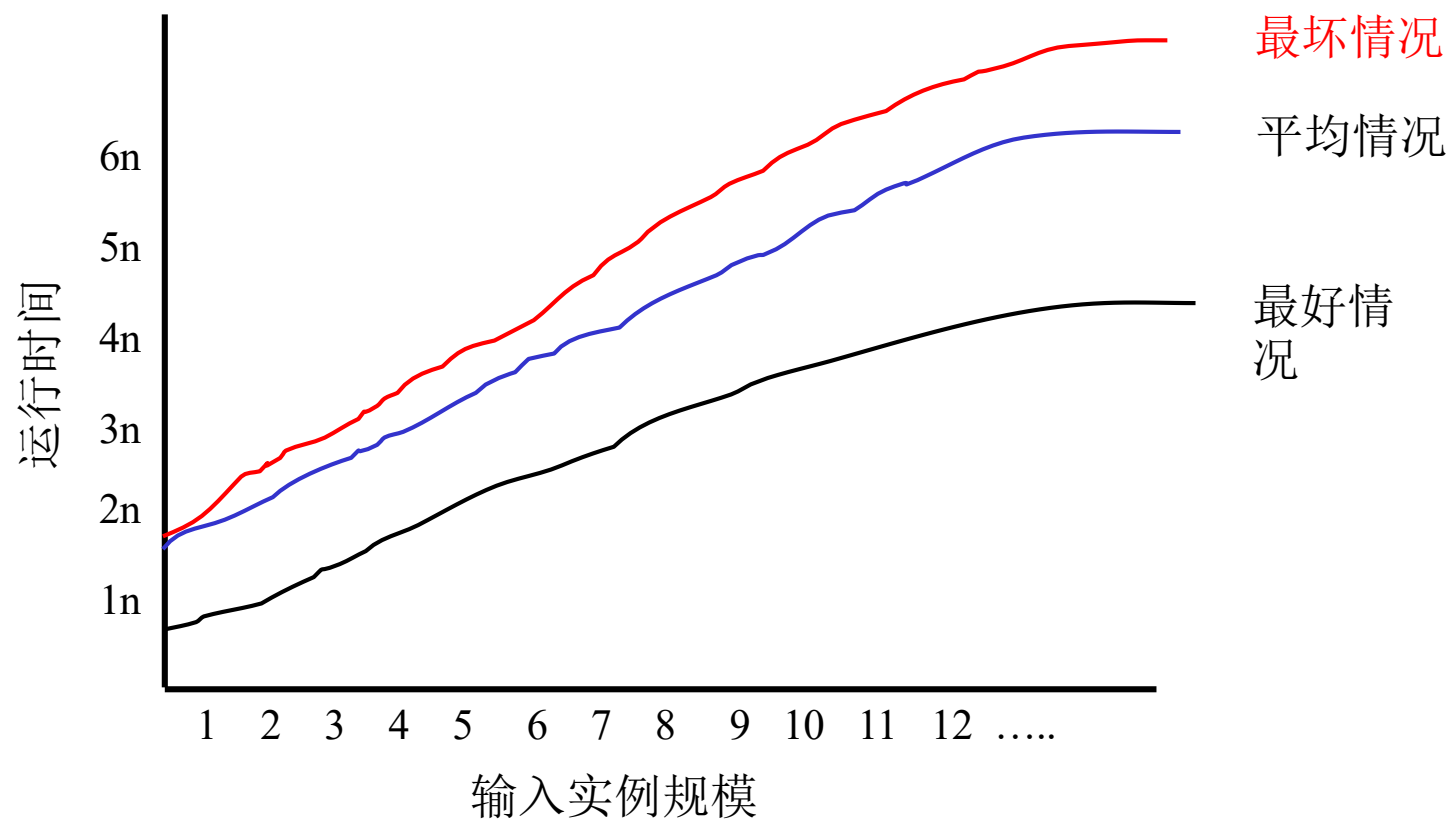
最好/最坏/平均情况 (2)

- 对于特定输入规模 n ，考察不同输入实例的运行时间：
- 在问题规模 n 下，算法的最坏情况时间是该算法在所有（可能无穷多个）规模为 n 的输入上的最大运行时间。



最好/最坏/平均情况 (3)

- 对各种规模的输入:



最好/最坏/平均情况 (4)

- **通常采用最坏情况：它是上界；在空管、手术等领域，知道最坏时间复杂度至关重要**
- 对某些算法，最坏情况其实经常发生
- **平均情况往往和最坏情况一样差**
- 求平均情况可能非常困难

二分查找的最坏时间

- 将下标落在 $[left, right]$ 内的整数称为活跃元素。
- 将每一次 while 循环称为一次迭代。每次迭代执行常数 c 次原子操作。一个宽松上界是 $c \leq 10$ 。
- 共有多少次迭代？

```
def binary_search(sorted_set, search_num):
    left = 0
    right = len(sorted_set) - 1

    while left <= right:
        middle = (left + right) // 2 # Find the middle index

        if sorted_set[middle] < search_num:
            left = middle + 1 # Narrow search to the right half
        elif sorted_set[middle] > search_num:
            right = middle - 1 # Narrow search to the left half
        else:
            return middle # Return the index if found

    return -1 # Return -1 if not found
```

二分查找的最坏时间

- 共有多少次迭代?
- **性质：第 i 次迭代后，活跃元素的个数至多为 $n/2^i$ 。**
- 假设总共有 h 次迭代。
 - 成立 $\frac{n}{2^h} \geq 1$ ，从而 $h \leq 1 + \log_2 n$
- 因此二分查找的最坏情况时间至多为
 - $10 \cdot (1 + \log_2 n)$
- 线性查找的最坏情况时间至多为
 - $4 \cdot (1 + n)$ (请自行验证)
- 当 n 较大时，二分查找远快于线性查找

代价分析练习 1：线性查找

算法 linearSearch(A, n, x):

```
for i ← 0 to n - 1 do
    if A[i] = x then
        return i
return -1
```

- 最好/平均/最坏代价分别是多少？

情况	代价
最好	1 次比较
最坏	n 次比较
平均	$(n+1)/2$ 次比较

代价分析练习 2：带条件的嵌套循环

算法 `mystery(A, n)`:

```
count ← 0
for i ← 0 to n - 1 do
    if A[i] > 0 then
        for j ← 0 to n - 1 do
            count ← count + 1
return count
```

最好/最坏代价?

什么输入会导致各自情况?

情况	代价
最好	(所有 $A[i] \leq 0$) $\Theta(n)$
最坏	(所有 $A[i] > 0$) $\Theta(n^2)$
平均	取决于数据分布

代价分析练习 3：前缀平均值

```
for i ← 0 to n - 1 do
    sum ← 0
    for j ← 0 to i do
        sum ← sum + X[j]
    Avg[i] = sum / (1+i)
```

最好/最坏代价

最好/最坏代价： $n(n+1)/2 = \Theta(n^2)$

渐近分析

- 考虑任意一个问题。假设算法 1 的运行时间为
 - $1000n \cdot c_{multi} + 10n \cdot c_{mem}$ time,
- 其中 c_{multi} 是一次乘法的时间, c_{mem} 是一次内存访问的时间; 算法 2 的运行时间为
 - $10n \cdot c_{multi} + 100n \cdot c_{mem}$ time.
- 哪个更好? 这取决于 c_{multi} 和 c_{mem} 的具体取值, 而它们因机器而异。

为什么不管常数？

- 考虑任意一个问题。假设算法 1 的运行时间为
 - $1000n \cdot c_{multi} + 10n \cdot c_{mem}$ time,
- 其中 c_{multi} 是一次乘法的时间, c_{mem} 是一次内存访问的时间; 算法 2 的运行时间为
 - $10n \cdot c_{multi} + 100n \cdot c_{mem}$ time.
- 在数学上, 我们希望得到在所有机器上都成立的普遍正确结论:
 - 无论何种机器, 它们的代价至多相差一个常数因子。

渐近分析

- **目标：去掉可能受具体实现和硬件影响的“细节”，以简化运行时间分析**
 - 类似于“取整”： $1,000,001 \approx 1,000,000$
 - $3n^2 \approx n^2$
- 抓住本质：
 - 运行时间随问题规模 n 的增长
 - 我们关心 n 很大时问题的效率（思考：为什么？）

渐近分析

- 抓住本质：
 - 运行时间随问题规模 n 的增长
 - 我们关心 n 很大时问题的效率（思考：为什么？）
- **例子：假设**
 - 算法 1 需要 n 次原子操作；
 - 而算法 2 需要 $10000 \cdot \log_2 n$ 次操作。
 - 当 $n = 2^{30}$ （大约 10^9 ）时，算法 2 快 $\frac{n}{10000 \cdot \log_2 n} > 3579$ 倍
 - 在理论计算机科学中，算法 2 被认为优于算法 1。

渐近记号

- “大 O” O-记号

- 渐近上界

- 设 $f(n)$ 和 $g(n)$ 为 n 的两个函数。

- 我们称 $f(n)$ 的渐近增长速度不高于 $g(n)$ ，若存在正常数 c 和 n_0 ，使得

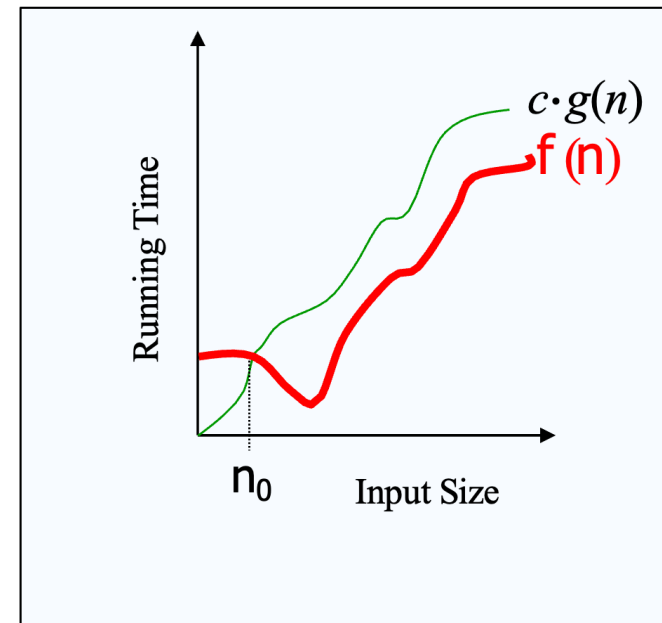
$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

- 可记作 $f(n) = O(g(n))$ 。

- 用于最坏情况分析

- 计算机科学的艺术：

- 在求解问题时尽量降低运行时间的增长速度

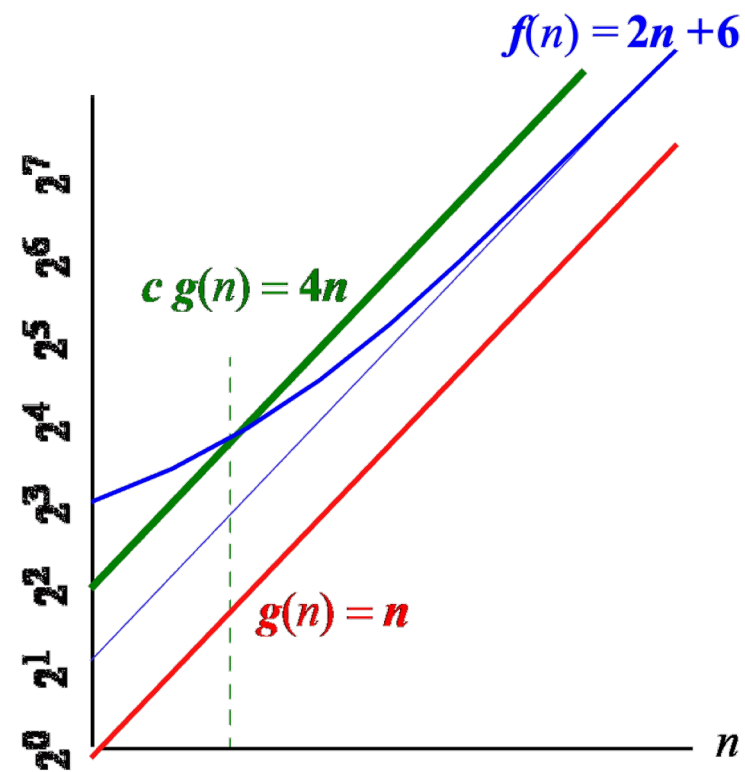


例子

对于函数 $f(n)$ 和 $g(n)$, 存在正常数 c 和 n_0 , 使得:
 $f(n) \leq c \cdot g(n)$ 对所有 $n \geq n_0$.

Conclusion:

$$2n + 6 = O(n).$$



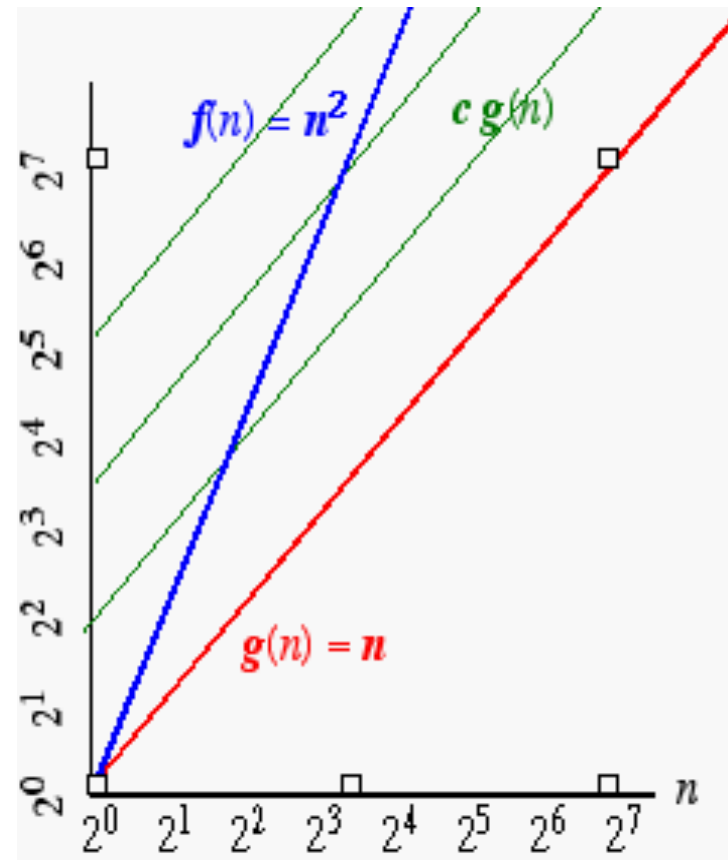
另一例子

另一方面……

n^2 不是 $O(n)$, 因为不存在 c 和 n_0 使得:

$$n^2 \leq cn \text{ for } n \geq n_0$$

右侧图说明: 无论选取多大的 c , 总存在足够大的 n , 使得 $n^2 > cn$ 。



例子

验证以下结论：

$$\begin{aligned}10000000 &= O(1) \\100\sqrt{n} + 10n &= O(n) \\1000n^{1.5} &= O(n^2) \\(\log_2 n)^3 &= O(\sqrt{n}) \\(\log_2 n)^{999999} &= O(n^{0.000001})\end{aligned}$$

全部为真

渐近记号

- **简单规则：去掉低阶项和常数因子。**

- $50n \log n$ is $O(n \log n)$
- $7n - 3$ is $O(n)$
- $8n^2 \log n + 5n^2 + n$ is $O(n^2 \log n)$

- **一个有趣的事实：**

$\log_{b_1} n = O(\log_{b_2} n)$ 对任意常数 $b_1 > 1$ 和 $b_2 > 1$ 成立。

- 因此，在计算机科学中，大 O 记号会省略所有常数对数底。例如，不写 $O(\log_2 n)$ ，而直接写 $O(\log n)$ 。

运行时间的渐近分析

- 用 O 记号将执行的基本操作次数表示为输入规模的函数。
- 比较渐近运行时间
 - 运行时间为 $O(n)$ 的算法优于运行时间为 $O(n^2)$ 的算法
 - 类似地, $O(\log n)$ 优于 $O(n)$
 - 函数的层次: $\log n < n < n^2 < n^3 < 2^n$
- 注意! 当心非常大的常数因子。运行时间为 $1,000,000 n$ 的算法仍是 $O(n)$, 但可能不如运行时间为 $2n^2$ 的算法高效, 后者是 $O(n^2)$

渐近分析示例

算法 prefixAverages1(X):

输入: 含 n 个元素的数组 X 。

输出: 含 n 个元素的数组 A , 其中 $A[i]$ 为 $X[0], \dots, X[i]$ 的平均值。

for $i \leftarrow 0$ **to** $n-1$ **do**

$a \leftarrow 0$

for $j \leftarrow 0$ **to** i **do**

$a \leftarrow a + X[j]$ $\longleftarrow$ 1 步

$A[i] \leftarrow a/(i+1)$

return 数组 A

$\left. \begin{array}{l} i \text{ 次迭代, } i = \\ 0, 1, 2, \dots, n-1 \end{array} \right\}$

$\left. \begin{array}{l} \text{ } \end{array} \right\} n \text{ 次迭代}$

分析: 运行时间为 $O(n^2)$

更好的算法

算法 prefixAverages2(X):

输入：由 n 个数组成的数组 X 。

输出：由 n 个数组成的数组 A ，其中 $A[i]$ 是元素 $X[0], \dots, X[i]$ 的平均值。

$s \leftarrow 0$

for $i \leftarrow 0$ **to** n **do**

$s \leftarrow s + X[i]$

$A[i] \leftarrow s/(i+1)$

return array A

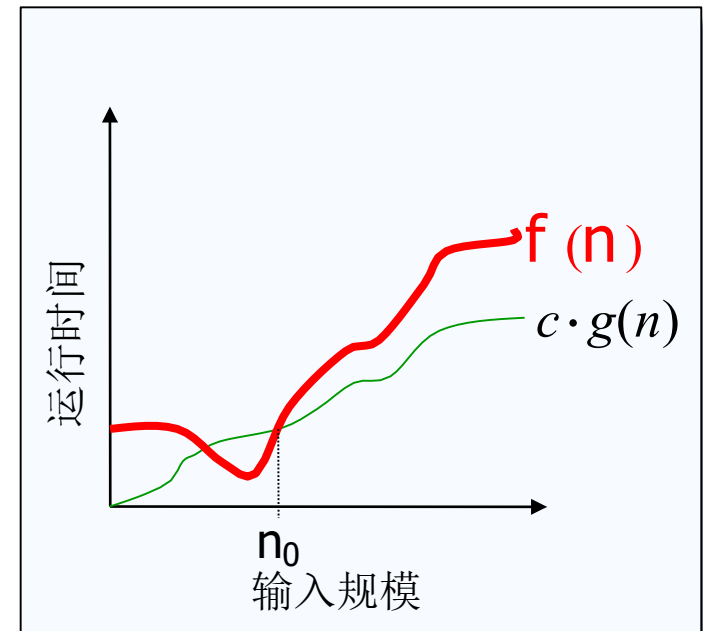
分析：运行时间为 $O(n)$

渐近记号（术语）

- 算法的特殊类别：
 - Logarithmic: $O(\log n)$
 - Linear: $O(n)$
 - Quadratic: $O(n^2)$
 - Polynomial: $O(n^k), k \geq 1$
 - Exponential: $O(a^n), a > 1$
- 大 O 的“亲戚”
 - $\Omega(f(n))$: 大 Omega——渐近下界
 - $\Theta(f(n))$: 大 Theta——渐近紧界

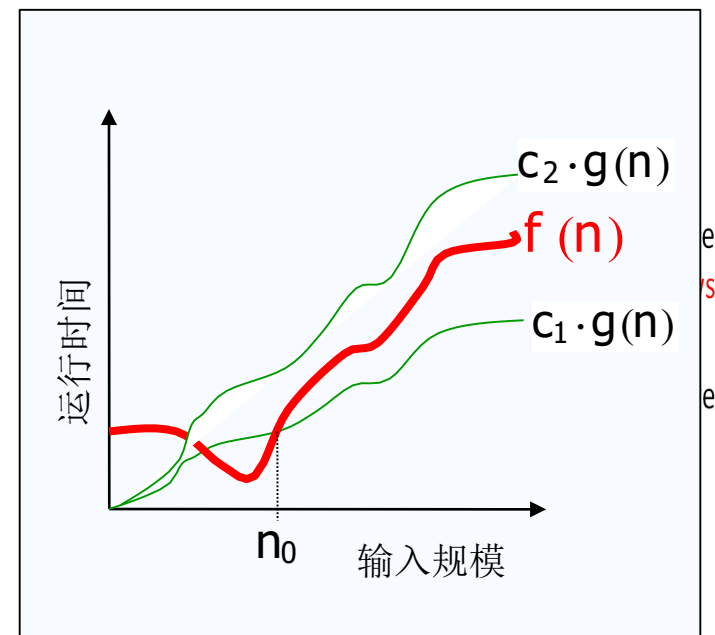
渐近记号

- “大 Omega” Ω -记号
 - 渐近下界
 - 设 $f(n)$ 和 $g(n)$ 为 n 的两个函数。
 - 我们称 $f(n)$ 的渐近增长速度不低于 $g(n)$ ，若存在正常数 c 和 n_0 ，使得
$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0.$$
 - 可记作 $f(n) = \Omega(g(n))$ 。
- 用于描述最好情况运行时间，或算法问题的下界
- 例如，在无序数组中查找的下界为 $\Omega(n)$ 。



渐近记号

- “大 Theta” Θ - 记号
 - 渐近紧界
 - 设 $f(n)$ 和 $g(n)$ 为 n 的两个函数。
 - 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则定义 $f(n) = \Theta(g(n))$ 表示 $f(n)$ 的渐近增长速度与 $g(n)$ 相同。
 - $f(n) = \Theta(g(n))$ 当且仅当存在正常数 c_1 、 c_2 和 n_0 , 使得
$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0.$$



例子

验证以下结论：

$$\log_2 n = \Omega(1)$$
$$n^{0.000001} = \Omega(\log_2 n)^{999999})$$

全部为真

$$\frac{2^n}{1000} = \Omega(n^{999999})$$

$$1000 + 30 \log_2 n + 1.5\sqrt{n} = \Theta(\sqrt{n})$$

$$n^2 + 2n + 1 = \Theta(n^2)$$

例子

验证以下结论：

$$\begin{array}{ll} n^2 = O(n \log n) & \text{假} \\ n + \log n = \Theta(\log n) & \text{假} \\ n \log n = O(n^2) & \text{真} \end{array}$$

渐近记号

另外两种渐近记号

- “小 o”记号 $f(n)$ 是大 O 的 $o(g(n))$ 非紧类比
 - 对每个 $c > 0$, 应存在 n_0 , 使得 $f(n) \leq c \cdot g(n)$ 对于 $n \geq n_0$
 - 用于比较运行时间。若 $f(n)$ 为 $o(g(n))$, 则称 $g(n)$ 支配 $f(n)$ 。
- “小 omega”记号 $f(n)$ 是大 Omega 的 $\omega(g(n))$ 非紧类比

渐近记号

- 与实数的类比

• $f(n)$ is $O(g(n))$	$\cong$	$f \leq g$
• $f(n)$ is $\Omega(g(n))$	$\cong$	$f \geq g$
• $f(n)$ is $\Theta(g(n))$	$\cong$	$f = g$
• $f(n)$ is $o(g(n))$	$\cong$	$f < g$
• $f(n)$ is $\omega(g(n))$	$\cong$	$f > g$

- 记号的滥用: $f(n) = O(g(n))$ 实际表示 $f(n) = \Theta(g(n))$

运行时间比较

Running Time	最大问题规模 (n)		
	1 second	1 minute	1 hour
$400n$	2500	150000	9000000
$20n \log n$	4096	166666	7826087
$2n^2$	707	5477	42426
n^4	31	88	244
2^n	19	25	31